



Entrega 5 - Prueba de Concepto: Cumplimiento de Criterios

GRUPO 16

SERGIO FERNANDO BARRERA MOLANO

202517034

HAROLD ANDRÉS BARTOLO MOSCOSO

202513889

JUAN JOSÉ RESTREPO BONILLA

202516633



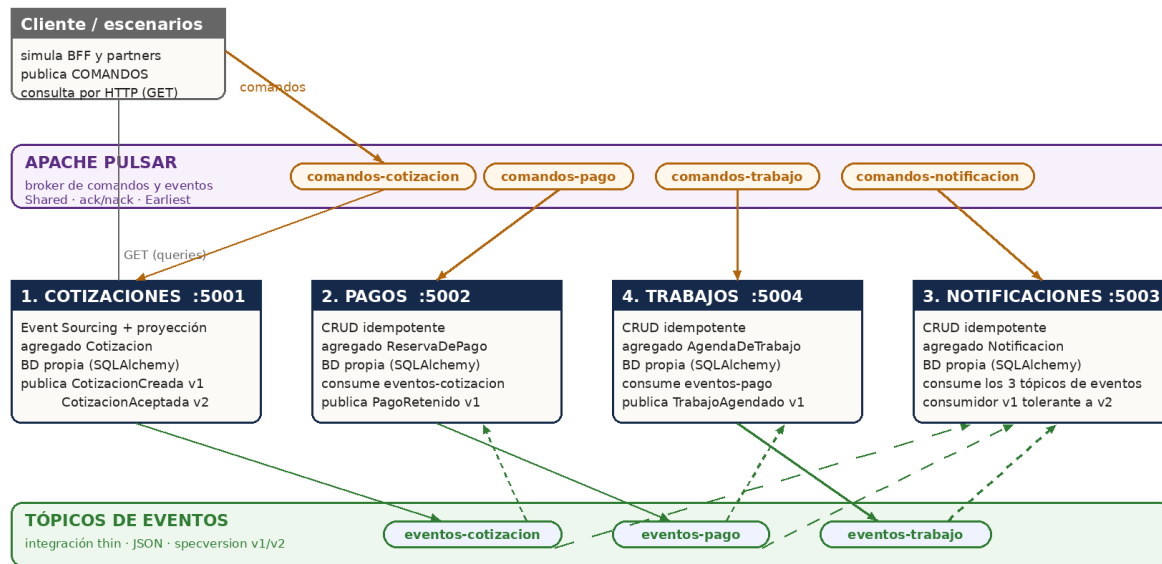
Entrega 4 ,Prueba de Concepto (POC): cumplimiento de los criterios de revisión

Contenido

1. Microservicios con comunicación por comandos y eventos sobre Apache Pulsar
2. Topología para la administración de datos: descentralizada
3. Modelo de datos por servicio: Event Sourcing y CRUD
4. Los tres escenarios de calidad y su coherencia con la Entrega 3
5. Experimentación: un escenario validado por atributo de calidad
6. Principios de DDD en el diseño
7. Anexo: ejecución de la POC y estructura del repositorio

Contexto y arquitectura de la POC

Hogar de los Alpes (HdA) migra de un monolito a un sistema reactivo por eventos para pasar de 12.000 a 36.000 trabajos/día con la expansión a México, Brasil y Argentina, 70% del volumen vía 30+ partners B2B2C y picos climáticos de 4x en 48 horas. La POC implementa la transacción larga central del negocio, **una cotización aceptada se convierte en un pago retenido y en un trabajo agendado**, con cuatro microservicios que se comunican exclusivamente por comandos y eventos a través de Apache Pulsar, y valida sobre ella un escenario de calidad por cada atributo priorizado en las entregas anteriores: escalabilidad (E1), modificabilidad (E6) y disponibilidad (E7).



Cada servicio es un paquete con seedwork + config (Flask-SQLAlchemy, UoW) + módulos/{dominio, aplicacion, infraestructura} + api.
Ningún servicio llama a otro por HTTP/gRPC. Topología de datos descentralizada: cuatro bases de datos, una por contexto acotado.

Figura 1. Vista de contenedores de la POC: cuatro servicios, tópicos de comandos (naranja) y de eventos (verde).

1. Microservicios con comunicación por comandos y eventos (Apache Pulsar)

Cumplimiento. Se desarrollaron cuatro microservicios en Python con Flask y Flask-SQLAlchemy siguiendo la estructura del tutorial 7 del curso, desplegados con Docker Compose junto a un clúster Apache Pulsar (imagen [apache/pulsar:3.2.4](#) en modo standalone, con healthcheck del que dependen los servicios). Toda la comunicación entre servicios es asíncrona por el broker; no existe ningún llamado HTTP ni gRPC entre servicios. Los endpoints HTTP GET que expone cada servicio son queries síncronas para clientes y para los escenarios de validación (excepción explícita de la rúbrica).



1.1 Tópicos de comandos y de eventos

Servicio	Tópico de comandos (consume)	Comandos	Tópicos de eventos que consume	Eventos que publica (tópico)
Cotizaciones (:5001)	comandos-cotizacion	CrearCotizacion, AceptarCotizacion	-	CotizacionCreada v1, CotizacionAceptada v2 (eventos-cotizacion)
Pagos (:5002)	comandos-pago	RetenerPago	eventos-cotizacion	PagoRetenido v1 (eventos-pago)
Trabajos (:5004)	comandos-trabajo	AgendarTrabajo	eventos-pago	TrabajoAgendado v1 (eventos-trabajo)
Notificaciones (:5003)	comandos-notificacion	RegistrarNotificacion	eventos-cotizacion, eventos-pago, eventos-trabajo	,(consumidor final)

Tabla 1. Tópicos de comandos y de eventos por servicio.

Comandos expresan una intención dirigida a un servicio y pueden rechazarse por reglas de negocio; **eventos** son hechos ya persistidos. Cada servicio tiene su propio tópico de comandos: al consumir un evento de integración de otro servicio, lo traduce a su comando de aplicación y lo ejecuta por el mismo camino (`ejecutar_commando`) que un comando recibido directamente. Esa simetría es la base de la saga orquestada de la Entrega 5. En esta entrega los servicios se oyen por los tópicos y reaccionan localmente, pero no se completa una transacción distribuida: cada uno confirma su parte y publica el hecho; la consistencia entre servicios es eventual.

1.2 Uso de Apache Pulsar

- El broker es un **puerto** del seedwork (`seedwork/infraestructura/broker.py`) con dos adaptadores intercambiables por configuración: **PulsarBroker** (entrega oficial) y **ArchivoBroker** (desarrollo sin Docker). Dominio y aplicación no conocen el broker.
- Un cliente Pulsar por proceso con un productor por tópico; suscripciones **Shared** (varios consumidores de una suscripción reparten carga) con `acknowledge` tras aplicar el efecto y `negative_acknowledge` ante error (reintento), e **Earliest** como posición inicial (una suscripción nueva no pierde lo publicado antes de crearse).
- Los consumidores corren en hilos del servicio (`infraestructura/consumidores.py`) y ejecutan cada mensaje dentro de un contexto de request de Flask para reutilizar la Unidad de Trabajo del tutorial.



- Los mensajes viajan en JSON con un envelope común: `id`, `time`, `specversion`, `type`, `class` (comando|evento), `service_source`, `data`.

```
[cotizaciones] COMANDO CrearCotizacion -> 7c848e3e-... (consumido de comandos-cotizacion)
[uow] evento CotizacionAceptada escrito en el EVENT STORE
[uow] señal 'CotizacionAceptadaIntegracion' emitida (post-commit)
[cotizaciones] EVENTO CotizacionAceptada v2 publicado en 'eventos-cotizacion'
[pagos] CONSUMIDO CotizacionAceptada v2 -> RetenerPago -> 25d91757-...
[pagos] EVENTO PagoRetenido v1 publicado en 'eventos-pago'
[trabajos] CONSUMIDO PagoRetenido v1 -> AgendarTrabajo -> d9849187-...
[notificaciones] CotizacionAceptada v2 recibida ,campo pais ignorado (consumidor v1)
```

Trazas reales de una corrida: el hecho viaja de servicio en servicio únicamente por los tópicos.

2. Topología para la administración de datos: descentralizada

Definición. Se adoptó la topología **descentralizada** (database per service): cada microservicio es dueño exclusivo de su base de datos y ningún servicio lee ni escribe la base de otro. Lo único compartido entre servicios es el contrato de mensajes.

2.1 Justificación

- Autonomía de despliegue y de evolución del esquema.** El dolor del monolito de la Entrega 1 era un esquema compartido que obligaba a coordinar a todos los equipos; con una BD por contexto acotado cada equipo cambia su modelo sin negociar (escenario E4 de la Entrega 3 y meta de 50 a 100 ingenieros).
- Escalado y tuning independientes.** Cotizaciones absorbe el pico climático con su propio almacenamiento sin competir con pagos o notificaciones (E1).
- Aislamiento de fallas.** La caída del servicio de pagos no bloquea ni corrompe los datos de cotizaciones: el núcleo sigue operando y el broker retiene los eventos (E7).
- Modelos distintos por contexto.** Cotizaciones necesita Event Sourcing (auditoría, replay); pagos y trabajos necesitan CRUD con unicidad; una BD compartida obligaría a un solo modelo para todos.

2.2 Tradeoffs asumidos y por qué no híbrida

No hay joins entre servicios: la composición se hace por eventos (cada consumidor recibe en el mensaje lo que necesita, p. ej. `PagoRetenido` lleva `id_trabajo` y `pais`) o por queries a cada servicio; la consistencia es eventual y hay más piezas operativas (cuatro bases, cuatro migraciones). Son exactamente los sacrificios que el árbol de



utilidad de la Entrega 2 declaró aceptables (consistencia inmediata y simplicidad operativa a cambio de escalabilidad, modificabilidad y disponibilidad). No se eligió la topología híbrida porque ningún par de servicios de este flujo comparte invariantes transaccionales que justifiquen una base común; agrupar, por ejemplo, pagos y trabajos reintroduciría el acoplamiento de esquema que se quiere eliminar.

2.3 Implementación

Servicio	Base de datos propia	Tablas	Quién la accede
Cotizaciones	cotizaciones.db	eventos_cotizacion (event store), cotizaciones (proyección)	solo cotizaciones
Pagos	pagos.db	reservas_pago, eventos_procesados	solo pagos
Trabajos	trabajos.db	agendas_trabajo, eventos_procesados	solo trabajos
Notificaciones	notificaciones.db	notificaciones, eventos_procesados	solo notificaciones

Tabla 2. Base de datos por servicio.

Cada servicio configura su conexión en `config/db.py` (Flask-SQLAlchemy; la URL se toma de `DB_URL`, lo que permite cambiar SQLite por PostgreSQL sin tocar código) y define sus modelos en `modulos/<bc>/infraestructura/dto.py`. No existe ningún import ni cadena de conexión cruzada entre servicios.

3. Modelo de datos por servicio: Event Sourcing y CRUD

Cumplimiento. Los cuatro servicios tienen base de datos (la rúbrica exige tres). Se definió e implementó **Event Sourcing** en el servicio núcleo y **CRUD** en los tres restantes, cada elección justificada por la naturaleza del contexto.



Servicio	Modelo	Cómo está implementado	Por qué
Cotizaciones	Event Sourcing + proyección	Tabla append-only <code>eventos_cotizacion</code> (seq, id, id_entidad, tipo, version, contenido) como fuente de verdad. El comando <code>AceptarCotizacion</code> reconstruye el agregado con <code>reconstruir</code> → <code>Cotizacion.desde_historia</code> antes de operar. La proyección <code>cotizaciones</code> (read model) se actualiza en la misma Unidad de Trabajo.	Hay dinero en juego: auditoría natural de cada cambio, posibilidad de reconstruir proyecciones nuevas por replay y separación C/Q del tutorial 7.
Pagos	CRUD idempotente	Tabla <code>reservas_pago</code> con unicidad por cotización + tabla <code>eventos_procesados</code> escrita en la misma transacción.	El escrow es estado simple con una invariante de unicidad; lo crítico es la idempotencia ante re-entregas (E7).
Trabajos	CRUD idempotente	Tabla <code>agendas_trabajo</code> + <code>eventos_procesados</code> .	Agenda confirmada del trabajo; regla: no se agenda sin pago retenido.
Notificaciones	CRUD idempotente	Tabla <code>notificaciones</code> + <code>eventos_procesados</code> .	Registro de salida; no requiere historia reconstruible.

Tabla 3. Modelo de datos por servicio.

3.1 Proyecciones y consultas con Event Sourcing

El enunciado pide mostrar cómo se manejan proyecciones y consultas cuando se usa Event Sourcing. En cotizaciones conviven las dos vistas: **escritura** por replay del event store (el handler carga el agregado aplicando su historia, valida la regla y hace append del evento nuevo) y **lectura** por la proyección materializada. Dos queries lo hacen visible:

- `GET /cotizaciones/<id>` lee la proyección (rápida, lo que consultan clientes y escenarios).
- `GET /cotizaciones/<id>/historia` devuelve la historia del event store y el agregado reconstruido por replay, demostrando que el estado es una función de sus eventos.

```
{
  "reconstruida_por_replay": {
    "estado": "ACEPTADA",
    "pais": "AR",
    "version": 2,
    ...
  },
  "historia": [
    {
      "seq": 1,
      "tipo": "CotizacionCreada",
      "version": "v1",
      ...
    },
    {
      "seq": 2,
      "tipo": "CotizacionAceptada",
      "version": "v2",
      ...
    }
  ]
}
```



3.2 Idempotencia como parte del modelo de datos

En los tres servicios CRUD, el handler del comando registra en la misma Unidad de Trabajo el efecto de negocio y la marca del mensaje consumido (puerto `RepositorioEventosProcesados`). Si el broker re-entrega un mensaje (semántica at-least-once), el handler lo detecta y lo ignora; el escenario E7 lo prueba activamente re-inyectando cinco eventos ya procesados.

4. Los tres escenarios de calidad y su coherencia con la Entrega 3

La Entrega 3 definió nueve escenarios, tres por atributo priorizado en la Entrega 2 (escalabilidad, modificabilidad, disponibilidad). Para la POC se eligió uno por atributo, con el criterio de que fuera relevante para el negocio (no un atributo fácil de probar), ejecutable a escala de POC y demostrativo de la expansión global.

Atributo	Escenario (Entrega 3)	Estímulo original	Medida original	Qué reproduce la POC
Escalabilidad	E1 ,Pico climático 4x en 48 h	Granizada: partners suben de ~290 a ~1.160 req/s de creación	0% rechazos; ACK p99 < 800 ms; cola drena < 6 h	Ráfaga de N comandos al tópico; el broker absorbe (buffering) y el flujo drena end-to-end sin rechazos ni duplicados; se mide throughput y tiempo de drenaje
Modificabilidad	E6 ,Evolución del contrato v1→v2	El productor publica una versión nueva de un evento con 8+ consumidores en producción	Downtime 0; consumidores rotos 0; convivencia v1/v2 ≥ 30 días	Cotizaciones publica CotizacionAceptada v2 (campo pais); notificaciones es consumidor v1; se publican v1 y v2 al mismo tópico y se verifica que ambos se procesan
Disponibilidad	E7 ,Caída de la pasarela de pagos 30 min	La pasarela responde 5xx en hora pico	Núcleo 100%; 0 pagos perdidos; 0 duplicados; recuperación < 5 min	Se mata el servicio pagos; sondas al núcleo durante la caída; Pulsar retiene; al volver se mide RTO, drenaje hasta trabajos y se re-inyectan duplicados

Tabla 4. Escenarios de calidad seleccionados y su reproducción en la POC.



4.1 Por qué estos tres

- **E1** es el escenario que da origen a toda la arquitectura reactiva: el negocio no puede rechazar siniestros durante una granizada. La POC no prueba "que Pulsar encola", sino el patrón completo buffering → autoescalado por lag → drenaje exacto con consumidores idempotentes.
- **E6** se prefirió sobre E4 (despliegue independiente) porque la rúbrica descarta explícitamente pruebas del tipo "se pudo agregar un método en menos de un día". La evolución de contratos con 30+ partners y la expansión a nuevos países (el campo `pais`) es la modificabilidad que importa al negocio.
- **E7** es el escenario crítico: protege la promesa "una fuga de agua no puede esperar". Es además el mismo tipo de experimento que sugiere la guía del curso (degradar un componente y verificar que se sigue operando).

4.2 Escala global

Los tres escenarios prueban que crecer a México, Brasil y Argentina no exige rearquitectura: el campo `pais` viaja en comandos y eventos, el objeto valor `Dinero` es multi-moneda (COP/MXN/BRL/ARS) por diseño, un país nuevo es dato más evolución compatible de contrato (E6), el pico de un país no bloquea a otro (E1) y la caída de una pasarela local no tumba el núcleo (E7), la misma tesis de E2 y E5 de la Entrega 3.

5. Experimentación: un escenario validado por cada atributo

Cada escenario es un script ejecutable en `escenarios/` que actúa como cliente (publica comandos y eventos al broker y consulta por HTTP), verifica criterios de éxito con aserciones y termina en **CUMPLIDO** o **FALLIDO**. Se ejecutaron contra Apache Pulsar en Docker (Modo A) y contra el adaptador de archivos (Modo B, runner `validar_todo.sh`). Resultado: **E1 = CUMPLIDO · E6 = CUMPLIDO · E7 = CUMPLIDO**.

5.1 E1 ,Escalabilidad

Publica N comandos `CrearCotizacion` en ráfaga, acepta cada cotización creada (un observador del tópico de eventos emite `AceptarCotizacion`) y espera a que pagos alcance exactamente N reservas.

```
== E1: ráfaga de 30 comandos CrearCotizacion (pico 4x a escala POC) ==
publicados 30 en 0.00s (52,298 comandos/s) ,0 rechazos (el broker encola)
drenaje end-to-end: 30 reservas en 1.0s · notificaciones acumuladas: 90
== E1 CUMPLIDO: 0 rechazos, drenaje completo, reservas exactas 30/30 (idempotencia) ==
```



Criterios: 0 rechazos en publicación; reservas == N exactamente (ni faltantes ni duplicados); throughput y tiempo de drenaje reportados. N se controla con la variable N (p. ej. N=2000 para la sustentación).

5.2 E6 ,Modificabilidad

Publica al tópico `eventos-cotizacion` un `CotizacionAceptada` v1 (sin país) y uno v2 (`pais=AR`), y verifica que el consumidor de notificaciones ,escrito contra v1 ,procesa ambos y registra la versión recibida.

```
== E6: publicando CotizacionAceptada v1 y v2 al mismo tópico ==  
v1 publicado (sin pais) · v2 publicado (pais=AR, expansión global)  
consumidor v1 procesó ambas: versiones nuevas registradas = ['v1', 'v2']  
== E6 CUMPLIDO: consumidores rotos = 0, convivencia v1/v2 en el tópico, downtime = 0 ==
```

Criterios: consumidores rotos = 0; ambas versiones conviven en el mismo tópico; downtime = 0. Es la verificación ejecutable de la política BACKWARD descrita en la sección 6.4.

5.3 E7 ,Disponibilidad (escenario crítico)

Se detiene el servicio pagos (SIGTERM al proceso en Modo B; `docker compose stop pagos` en Modo A) y se verifican cuatro mediciones, no una afirmación:

- **Disponibilidad del núcleo durante la caída:** 10 sondas HTTP a cotizaciones mientras pagos está muerto → 100%; además el núcleo crea y acepta N cotizaciones y publica sus eventos.
- **RPO = 0:** al rearrancar pagos, el broker le entrega los N eventos retenidos; reservas N/N, perdidos 0.
- **RTO** medido desde el re arranque hasta el drenaje completo.
- **Duplicados = 0 probado activamente:** se re-inyectan 5 copias exactas de eventos ya procesados (re-entrega at-least-once) y pagos y trabajos los ignoran. La cadena se cierra: cada pago retenido termina en un trabajo agendado (N/N).



```

== E7 [1/4] deteniendo PAGOS (simula caída de la pasarela 30 min) ==
pagos: caído (sin respuesta HTTP)
núcleo: 15/15 aceptadas y publicadas · disponibilidad 100% (10/10 sondas) · pagos sigue caído: True
== E7 [2/4] rearrancando PAGOS ,el broker debe entregar TODO lo retenido ==
RTO = 0.5s hasta drenar · reservas 15/15 · perdidos = 0
== E7 [3/4] cierre del ciclo: PagoRetenido -> trabajo AGENDADO ==
trabajos agendados 15/15 (cada pago retenido terminó en un trabajo)
== E7 [4/4] re-inyectando 5 eventos YA procesados (re-entrega at-least-once) ==
tras la re-entrega: reservas 15/15, trabajos 15/15 -> duplicados en pagos = 0, en trabajos = 0
== E7 CUMPLIDO: núcleo 100% disponible durante la caída · perdidos = 0 (RPO=0) · RTO = 0.5s ·
duplicados = 0 con 5 re-entregas inyectadas · cadena cerrada 15/15 ==

# evidencia en el log del pagos relanzado:
[pagos] duplicado 7c848e3e IGNORADO (idempotencia) (x5)

```

Tácticas validadas: desacople por broker (pagos es consumidor, no dependencia síncrona del núcleo), retención y re-entrega del broker, consumidores idempotentes y eventos autosuficientes. En producción el RTO crece con el backlog; por eso la Entrega 3 fija recuperación < 5 min y autoescalado por lag.

6. Principios de DDD en el diseño

Los cuatro servicios comparten la anatomía del tutorial 7 del curso; DDD no es una capa decorativa sino la estructura del código.

```

src/<servicio>/
  seedwork/          Entidad, AgregacionRaiz, EventoDominio, ObjetoValor, ReglaNegocio +
ValidarReglasMixin,
  dominio/           Fabrica, Repositorio y Mapeador (puertos), Comando/Query +
ejecutar_comando/ejecutar_query
  aplicacion/        (singledispatch), UnidadTrabajo + UnidadTrabajoPuerto (señales pydispatch), broker
(puerto)
  infraestructura/
  config/db.py, config/uow.py      Flask-SQLAlchemy y UnidadTrabajoSQLAlchemy
  modulos/<bc>/
    dominio/         entidades · objetos_valor · eventos · reglas · fabricas · repositorios (puertos) ·
excepciones
    aplicacion/      comandos/ · queries/ · dto · mapeadores · handlers (señales *Integracion)
    infraestructura/ dto (modelos SQLAlchemy) · repositorios · mapeadores · fabricas · despachadores ·
consumidores · schema/v1 (y v2) eventos de integración
  api/               blueprint Flask (solo queries) · create_app
  main.py

```

6.1 Contextos acotados

Cada microservicio es un contexto acotado de la Entrega 1 con su propio lenguaje, modelo, base de datos y tópico de comandos: **Cotizaciones** (núcleo), **Pagos** (escrow), **Gestión de Trabajos** (núcleo) y **Notificaciones** (soporte).



Las relaciones entre contextos son Published Language por eventos de integración versionados; los esquemas (`schema/v1`, `schema/v2`) actúan como Shared Kernel copiado y versionado en cada servicio.

6.2 Agregaciones, entidades, objetos valor y reglas

Servicio	Raíz de agregación	Objetos valor	Reglas de negocio	Evento de dominio
Cotizaciones	Cotizacion (event-sourced: aplicar / desde_historia)	Dinero(monto, Moneda), EstadoCotizacion	MontoDebeSerPositivo, SoloEmitidaSePuedeAceptar	CotizacionCreada, CotizacionAceptada
Pagos	ReservaDePago (fábrica retener)	Dinero, Estado	MontoDebeSerPositivo	PagoRetenido
Trabajos	AgendaDeTrabajo (fábrica agendar)	Estado	DebeExistirPagoRetenido	TrabajoAgendado
Notificaciones	Notificacion (fábrica registrar)	Estado	DestinatarioObligatorio	NotificacionRegistrada

Tabla 5. Agregaciones, objetos valor, reglas y eventos de dominio por servicio.

Las raíces heredan de `AgregacionRaiz` y acumulan eventos de dominio; las fábricas (`dominio/fabricas.py`) validan las reglas al construir (`validar_regla`); los objetos valor son `dataclass(frozen=True)` sin identidad.

6.3 Capas, arquitectura cebolla e inversión de dependencias

- **Dominio** no importa Flask, SQLAlchemy ni el broker: solo seedwork y sus propios módulos.
- **Aplicación** orquesta: el handler de un comando construye o reconstruye el agregado, invoca su comportamiento y registra los batches en la Unidad de Trabajo; las queries devuelven DTOs (CQS).
- **Infraestructura** implementa los puertos: repositorios sobre Flask-SQLAlchemy, mapeadores entidad↔modelo y dominio→integración, despachadores y consumidores del broker.
- **Inversión de dependencias:** `RepositorioCotizaciones`, `RepositorioEventosCotizaciones` y `RepositorioEventosProcesados` son interfaces del dominio; `FabricaRepositorio` entrega la implementación. El broker es un puerto con adaptadores Pulsar/archivo elegidos por variable de entorno.
- **Unidad de Trabajo (tutorial 7):** `registrar_batch + commit`; la señal `<Evento>Dominio` se emite pre-commit (y escribe el event store en cotizaciones) y `<Evento>Integracion` post-commit → despachador → tópico. Un mismo commit contiene el efecto de negocio y la marca de idempotencia.



6.4 Tipos de eventos, formato y versionamiento de esquemas

Eventos de integración thin (llaves y datos esenciales) y no de carga de estado: los consumidores de este flujo solo necesitan llave, monto, trabajo y país; un fat event acoplaría su esquema a todos. **Formato JSON** con versionado propio en el envelope (`specversion` por mensaje): legible en logs y tópicos, sin toolchain de compilación ni dependencias nativas en cuatro servicios, y permite que v1 y v2 convivan en el mismo tópico (a diferencia de versionar por nombre de tópico). **Política BACKWARD**: una versión nueva solo agrega campos opcionales (v2 de `CotizacionAceptada` añade `pais`); nunca renombra ni elimina; un cambio incompatible exige un tipo de evento nuevo. Tradeoff asumido: sin validación automática de esquema en el broker (que daría Avro con Schema Registry); migrar a Avro es un adaptador más, no un cambio de arquitectura.

Anexo. Ejecución de la POC

Modo A ,Apache Pulsar con Docker (entrega oficial; Windows, Mac y Linux)

```
docker compose up --build -d          # Pulsar + 4 microservicios
curl localhost:5001/health ... localhost:5004/health
docker compose build escenarios       # contenedor cliente de escenarios (una vez)
docker compose run --rm escenarios python escenario_e1_escalabilidad.py
docker compose run --rm escenarios python escenario_e6_modificabilidad.py
bash escenarios/e7_docker.sh          # stop pagos -> fase caida -> start pagos -> fase
recuperacion
docker compose logs -f pagos          # trazas [uow] / [pagos] en vivo
docker compose down -v
```

Los escenarios corren en un contenedor porque `pulsar-client` de Python no publica wheels para Windows; el contenedor comparte la red de compose con el broker y los servicios.

Modo B ,sin Docker (adaptador de archivos, mismo código)

```
bash escenarios/validar_todo.sh        # levanta los 4 servicios, corre E1+E6+E7 y apaga todo
# o manualmente, una terminal por servicio:
export BROKER=archivo BROKER_DIR=$PWD/broker_dev
cd servicios/cotizaciones && PYTHONPATH=src python src/cotizaciones/main.py # y así con los otros tres
```



Despliegue

Contenedores Docker (uno por microservicio más Pulsar) orquestados con Docker Compose sobre una VM Linux de Google Cloud Compute Engine, idénticos a los que corren en Docker Desktop. Un contenedor por servicio es la unidad de despliegue independiente que exige E4; las mismas imágenes pasan a GKE cuando el volumen de la expansión global lo requiera, cambiando únicamente `BROKER_HOST`.

Estructura del repositorio

```
entrega4-hogar-de-los-alpes/
├── README.md · docker-compose.yml · .gitignore
├── servicios/{cotizaciones, pagos, notificaciones, trabajos}/
│   ├── Dockerfile · requirements.txt · src/<servicio>/{seedwork, config, modulos, api, main.py}
└── escenarios/
    ├── contratos.py · Dockerfile · e7_docker.sh · validar_todo.sh
    ├── escenario_e1_escalabilidad.py · escenario_e6_modificabilidad.py · escenario_e7_disponibilidad.py
```



© - **Derechos Reservados:** la presente obra, y en general todos sus contenidos, se encuentran protegidos por las normas internacionales y nacionales vigentes sobre propiedad Intelectual, por lo tanto su utilización parcial o total, reproducción, comunicación pública, transformación, distribución, alquiler, préstamo público e importación, total o parcial, en todo o en parte, en formato impreso o digital y en cualquier formato conocido o por conocer, se encuentran prohibidos, y solo serán lícitos en la medida en que se cuente con la autorización previa y expresa por escrito de la Universidad de los Andes.

De igual manera, la utilización de la imagen de las personas, docentes o estudiantes, sin su previa autorización está expresamente prohibida. En caso de incumplirse con lo mencionado, se procederá de conformidad con los reglamentos y políticas de la universidad, sin perjuicio de las demás acciones legales aplicables.